

JavaScript

Guía completa de estudio · Sesión 3

Esta guía cubre los fundamentos del lenguaje: su historia, variables, tipos de datos, estructuras de control y entrada de datos por teclado. Cada concepto incluye definición técnica, ejemplos reales y un ejercicio inmediato para aplicarlo.

SECCIÓN 1 · HISTORIA Y CONTEXTO

Historia y contexto de JavaScript

JavaScript nació en **1995**. Brendan Eich lo diseñó en **10 días** mientras trabajaba en Netscape con un objetivo simple: agregar comportamiento a páginas web que hasta ese momento eran documentos estáticos. Se llamó primero Mocha, luego LiveScript, y finalmente JavaScript — un nombre de marketing que aprovechó la popularidad de Java en esa época. No tienen ninguna relación técnica.

En **1997** fue estandarizado como **ECMAScript** por ECMA International. La versión más importante de la historia moderna fue **ES6 (2015)**: introdujo **let**, **const**, arrow functions, template literals, clases y módulos. Desde entonces el estándar se actualiza cada año.

DEF **JavaScript** es un lenguaje interpretado, de tipado dinámico y débil, basado en ECMAScript (ECMA-262). Su código fuente es analizado y ejecutado en tiempo real por un motor especializado (V8 en Chrome/Node, SpiderMonkey en Firefox, JavaScriptCore en Safari), sin requerir compilación previa. — *ECMAScript 2023 Language Specification, §4*

Motores de JavaScript

Motor	Dónde corre	Empresa
V8	Chrome, Edge, Node.js, Deno	Google
SpiderMonkey	Firefox	Mozilla
JavaScriptCore	Safari	Apple

DEF **Node.js** es un entorno de ejecución de JavaScript construido sobre el motor V8 que permite correr JS fuera del navegador, con acceso a sistema de archivos, red y terminal. — *nodejs.org/en/about*

SECCIÓN 2 · INSTALACIÓN DE NODE.JS

Instalación de Node.js

Para ejecutar JavaScript en la terminal de VS Code necesitas Node.js instalado. Es lo que convierte tu computador en un entorno capaz de correr JS sin un navegador.

Paso 1 — Descargar e instalar

Ve a nodejs.org → descarga la versión **LTS** (Long Term Support). Instálala con las opciones por defecto.

Paso 2 — Verificar en la terminal de VS Code

Abre la terminal con **Ctrl + `** y ejecuta:

```
node --version

npm --version

# Deberías ver algo como:

# v20.11.0

# 10.2.4
```

Paso 3 — Tu primer archivo JS

Crea un archivo **hola.js**, escribe el código, guárdalo y ejecútalo desde la terminal:

```
// hola.js

console.log("Hola, JavaScript");

console.log(2 + 3);

console.log(typeof "texto");
```

```
node hola.js

# Hola, JavaScript

# 5

# string
```

- **Debes estar en la carpeta del archivo.** Usa **cd nombre-carpeta** para navegar. Si el archivo está en el escritorio: **cd Desktop** (Mac) o **cd Escritorio** (Windows). Comprueba con **ls** (Mac/Linux) o **dir** (Windows) que el archivo aparece en la lista.

— Ejercicio rápido 1

Crea el archivo `hola.js`, agrega tres `console.log()` con contenidos diferentes (un número, una frase y una operación matemática) y ejecútalo con `node`. Observa el resultado de cada uno en la terminal.

Pista: `console.log()` acepta cualquier valor — número, texto, operación directa

SECCIÓN 3 · VARIABLES

Variables: let, const y el problema de var

DEF

Variable: referencia con nombre almacenada en memoria que asocia un identificador a un valor. El tipo de declaración determina si puede reasignarse, en qué ámbito existe y cuándo puede usarse. — *MDN — Grammar and types: Declarations*

Scope y Hoisting — los dos conceptos que cambian todo

DEF

Scope (ámbito): región del código donde una variable existe y es accesible. **let** y **const** tienen scope de bloque — solo viven dentro del `{ }` donde se declararon. **var** tiene scope de función o global — ignora los bloques `{ }` intermedios. — *MDN — let: block scope*

DEF

Hoisting: comportamiento del intérprete que mueve declaraciones al inicio de su scope antes de ejecutar. **var** se inicializa como **undefined** al ser hoisted. **let** y **const** quedan en la Temporal Dead Zone (TDZ): cualquier acceso antes de su declaración lanza **ReferenceError**. — *MDN Glossary — Hoisting*

Keyword	¿Reassignable?	Scope	Hoisting	Usar
let	Sí	Bloque { }	TDZ (ReferenceError si acceso previo)	■
const	No	Bloque { }	TDZ (ReferenceError si acceso previo)	■
var	Sí	Función o global	undefined (sin error — peligroso)	■

let — variable que puede cambiar de valor

```
let puntaje = 0;

puntaje = puntaje + 10;

puntaje += 5;

console.log("Puntaje:", puntaje);

# Puntaje: 15

let ciudad = 'Cali';

ciudad = 'Medellín';

console.log("Ciudad actual:", ciudad);

# Ciudad actual: Medellín
```

const — referencia que no cambia

```
const IVA = 0.19;

const NOMBRE_EMPRESA = 'MiTienda';

let precio = 50000;

let total = precio + (precio * IVA);

console.log(`Total con IVA: ${total}`);

# Total con IVA: $59500

# IVA = 0.21;

# → TypeError: Assignment to constant variable
```

var — por qué produce bugs silenciosos

El problema no es que **var** no funcione — el problema es que funciona pero produce resultados inesperados sin ningún aviso de error:

```
# ■ Con var – bug silencioso

var stock = 100;

if (true) {

    var stock = 0;

    # Intención: variable local al bloque

}

console.log("Stock:", stock);

# Stock: 0 ← se sobrescribió sin aviso
```

```
# ■ Con let – comportamiento predecible

let inventario = 100;

if (true) {

    let inventario = 0;

    # Variable NUEVA, local al bloque {}

}
```

```
}

console.log("Inventario:", inventario);

# Inventario: 100 ← la exterior no cambió
```

- Muchos tutoriales en internet todavía usan **var**. Lo van a encontrar. El problema: el bug ocurre sin ningún mensaje de error — el programa simplemente imprime un valor equivocado. Regla: **let** para lo que cambia, **const** para lo que no. **var** no.

— Ejercicio rápido 2

Declara tres variables: una con `let` (nombre de un producto), una con `const` (precio fijo) y una con `let` (cantidad en inventario). Calcula e imprime el valor total del inventario (precio × cantidad) usando template literals en el formato: 'Total en inventario: \$[valor]'.

Pista: El valor total se calcula multiplicando dentro del template literal: `${precio * cantidad}`

SECCIÓN 4 · TIPOS DE DATOS

Tipos de datos primitivos

DEF **Tipo de dato primitivo:** valor inmutable que no es un objeto y no tiene métodos propios. JS tiene 7 tipos primitivos. Son inmutables: las operaciones producen un nuevo valor, no modifican el original. — *MDN* — *JavaScript data types and data structures*

Tipo	Ejemplo	typeof	Diferencia con Python
number	42, 3.14, -7, Infinity	"number"	int + float en un solo tipo
string	"hola", `template`	"string"	igual que str
boolean	true, false	"boolean"	True/False (mayúscula en Python)
undefined	var sin asignar	"undefined"	no existe equivalente directo
null	null	"object" ■	equivale a None
bigint	9007199254740991n	"bigint"	int ilimitado de Python
symbol	Symbol("id")	"symbol"	no existe en Python

number — un solo tipo para todos los números

En Python **type(10)** devuelve **int** y **type(10.0)** devuelve **float**. En JS no existe esa distinción — todo número es **number**, usando el estándar IEEE 754 de doble precisión:

```
let unidades = 5;

let precio_unitario = 12500.50;

let descuento = 0.10;

console.log(typeof unidades);

# number ← entero

console.log(typeof precio_unitario);

# number ← decimal — MISMO tipo

let subtotal = unidades * precio_unitario;

let total = subtotal - (subtotal * descuento);

console.log(`Total: ${total}`);

# Total: $56252.25
```

string — tres formas, una recomendada

```
let s1 = "Hola mundo";

let s2 = 'Hola mundo';

let s3 = `Hola mundo`;

# Template literal — la más potente

let producto = 'Teclado mecánico';

let precio = 280000;

let iva = precio * 0.19;

console.log(`${producto} → precio: ${precio}, IVA: ${iva}`);

# Teclado mecánico → precio: $280000, IVA: $53200

# Concatenación clásica — más verbosa

console.log(producto + " cuesta $" + precio);
```

null, undefined y el bug histórico de JS

```
let pedido = null;

# null = ausencia intencional de valor

console.log(typeof pedido);

# object ← bug histórico desde 1995

console.log(pedido === null);

# true ← así se verifica correctamente

let direccion;

# undefined = variable declarada sin valor

console.log(typeof direccion);

# undefined

console.log(direccion === undefined);

# true
```


- **typeof null devuelve 'object'** — es un error del diseño original mantenido por retrocompatibilidad. Para verificar si algo es null, siempre usa **=== null**. Para undefined, usa **=== undefined** o simplemente verifica si el valor es falsy.

— Ejercicio rápido 3

Declara estas variables: nombre de un cliente (string), saldo de su cuenta (number con decimales), cuenta activa (boolean), dirección pendiente de llenar (null). Imprime cada una con su typeof en el formato: 'campo → valor: X, tipo: Y'. ¿Cuál te da un tipo inesperado?

Pista: typeof null da 'object' — no es un error tuyo, es un bug histórico de JS

SECCIÓN 5 · ESTRUCTURAS DE CONTROL

Estructuras de control

DEF **Estructura de control:** instrucción que altera el flujo secuencial de ejecución, redirigiendo qué instrucciones corren a continuación según una condición, o repitiendo un bloque mientras se cumpla una condición. —

Sebesta, Concepts of Programming Languages, 11a ed., cap. 8

Elemento	Python	JavaScript
Bloque de código	Indentación	Llaves {} obligatorias
Condición	Sin paréntesis	Paréntesis () obligatorios
Incremento	<code>i += 1</code>	<code>i++</code> o <code>i += 1</code>
for clásico	<code>for i in range(n)</code>	<code>for (let i=0; i<n; i++)</code>
for sobre lista	<code>for x in lista</code>	<code>for (let x of lista)</code>
Igualdad segura	<code>==</code>	<code>===</code> (valor Y tipo — usar siempre)

if / else if / else

DEF **if:** evalúa la expresión entre paréntesis. Si es true o truthy, ejecuta el bloque. **else if** encadena condiciones adicionales evaluadas en orden. **else** captura todos los casos que no cumplieron ninguna condición anterior. —

MDN — if...else

Ejemplo — sistema de descuentos según monto de compra:

```
let monto_compra = 750000;

let descuento = 0;

if (monto_compra >= 1000000) {

    descuento = 0.20;

    # 20% para compras >= $1.000.000

} else if (monto_compra >= 500000) {

    descuento = 0.10;

    # 10% para compras entre $500.000 y $999.999

} else if (monto_compra >= 200000) {

    descuento = 0.05;

    # 5% para compras entre $200.000 y $499.999

}
```

```
} else {  
  
    descuento = 0;  
  
    # Sin descuento  
  
}  
  
let valor_descuento = monto_compra * descuento;  
  
let total = monto_compra - valor_descuento;  
  
console.log(`Descuento: ${descuento * 100}%`);  
  
console.log(`Total a pagar: ${total}`);  
  
# Descuento: 10%  
  
# Total a pagar: $675000
```

→ **== vs ===** — Usa siempre **===**. Con **==** JavaScript intenta convertir los tipos antes de comparar: **0 == false** da true, **" == false** da true, **null == undefined** da true. Con **===** compara valor y tipo sin conversión: **0 === false** da false.

— Ejercicio rápido 4

Declara una variable `temperatura_cpu` (número). Usa `if/else if/else` para imprimir: 'Normal' (menor de 60°), 'Precaución' (60 a 79°), 'Crítico' (80 a 89°) o 'Apaga el equipo' (90° o más). Prueba con al menos tres valores distintos.

Pista: Para el rango del medio usa `&&` para combinar dos condiciones

while

DEF **while:** evalúa su condición antes de cada iteración. Si es true, ejecuta el bloque y vuelve a evaluar. El ciclo termina cuando la condición es false. Si la condición nunca cambia, el ciclo es infinito. — *MDN* — *while*

Ejemplo — simulación de retiro en cajero automático:

```
let saldo = 500000;

let retiro = 80000;

let intentos = 0;

while (saldo >= retiro) {

    saldo = saldo - retiro;

    intentos += 1;

    console.log(`Retiro #${intentos} → Saldo: ${saldo}`);

}

console.log(`Total de retiros: ${intentos}`);

# Retiro #1 → Saldo: $420000

# Retiro #2 → Saldo: $340000

# Retiro #3 → Saldo: $260000

# Retiro #4 → Saldo: $180000

# Retiro #5 → Saldo: $100000

# Retiro #6 → Saldo: $20000

# Total de retiros: 6
```

- Si olvidas modificar la variable de control dentro del while el programa nunca termina. Para detener un proceso colgado en VS Code usa **Ctrl + C** en la terminal.

— Ejercicio rápido 5

Una cuenta de ahorros tiene \$1.200.000 y cobra un mantenimiento mensual de \$35.000. Usa while para simular cuántos meses tarda la cuenta en quedar con menos de \$35.000. Imprime el saldo al final de cada mes y al terminar muestra el total de meses.

Pista: El while corre mientras el saldo sea >= al costo de mantenimiento

for clásico

DEF **for clásico:** agrupa tres expresiones: inicialización (ejecutada una vez), condición (evaluada antes de cada vuelta) y actualización (ejecutada al final de cada vuelta). El bloque se repite mientras la condición sea true. — *MDN — for*

Ejemplo — tabla de precios con IVA para diferentes cantidades:

```
const PRECIO_BASE = 25000;

const IVA = 0.19;

for (let cantidad = 1; cantidad <= 5; cantidad++) {

  let subtotal = PRECIO_BASE * cantidad;

  let total = subtotal + (subtotal * IVA);

  console.log(`${cantidad} ud(s) → subtotal: ${subtotal}, total: ${total}`);

}

# 1 ud(s) → subtotal: $25000, total: $29750
# 2 ud(s) → subtotal: $50000, total: $59500
# 3 ud(s) → subtotal: $75000, total: $89250
# 4 ud(s) → subtotal: $100000, total: $119000
# 5 ud(s) → subtotal: $125000, total: $148750
```

— Ejercicio rápido 6

Una empresa paga a sus empleados un salario base de \$2.500.000. Usa un for para imprimir la nómina de 6 meses: mes, salario y un bono del 10% que se suma solo en los meses 3 y 6. Formato: 'Mes [N]: salario \$X, total a pagar: \$Y'.
Pista: Dentro del for usa un if para detectar si el mes actual es 3 o 6 con el operador %

for...of

DEF **for...of:** itera sobre los valores de cualquier iterable (arrays, strings). Asigna el valor de cada elemento a una variable de bloque en cada vuelta. No expone el índice — solo el valor. — *MDN — for...of*

Ejemplo — revisión de stock de productos:

```
let productos = [

  { nombre: "Laptop",    stock: 3 },

  { nombre: "Mouse",     stock: 0 },
```

```
{ nombre: "Monitor", stock: 12 },  
  
{ nombre: "Teclado", stock: 0 },  
  
{ nombre: "Webcam", stock: 5 },  
  
];  
  
for (let p of productos) {  
  if (p.stock === 0) {  
    console.log(`■ ${p.nombre}: SIN STOCK`);  
  } else {  
    console.log(`✓ ${p.nombre}: ${p.stock} unidades`);  
  }  
}
```

✓ Laptop: 3 unidades

■ Mouse: SIN STOCK

✓ Monitor: 12 unidades

■ Teclado: SIN STOCK

✓ Webcam: 5 unidades

→ **for clásico vs for...of:** usa el for clásico cuando necesitas el índice o control preciso del rango (del 1 al 10, de 2 en 2, de atrás para adelante). Usa for...of cuando solo necesitas los valores de una lista.

⇒ Ejercicio rápido 7

Tienes una lista de calificaciones: [72, 88, 45, 91, 60, 55, 78]. Usa for...of para recorrerlas e imprimir cada una con su estado: 'Aprobó' (≥ 60) o 'Reprobó' (< 60). Al terminar imprime cuántos aprobaron y cuántos reprobaron.

Pista: Declara dos contadores antes del for e increméntalos dentro del if/else

SECCIÓN 6 · ENTRADA DE DATOS POR TECLADO

Entrada de datos por teclado con prompt-sync

El método **prompt()** que conocen del navegador no existe en Node.js — es una función del objeto **window**, exclusivo del entorno del browser. Para tener la misma experiencia en la terminal de VS Code usamos un paquete llamado **prompt-sync**, que replica exactamente ese comportamiento.

DEF **prompt-sync** es un paquete de Node.js que expone una función síncrona para leer una línea de la entrada estándar (teclado). Bloquea la ejecución hasta que el usuario presiona Enter y devuelve el valor escrito como string. — npmjs.com/package/prompt-sync

Instalación — una sola vez por proyecto

En la terminal de VS Code, dentro de la carpeta de tu proyecto:

```
npm init -y

# Crea el archivo package.json en tu carpeta

npm install prompt-sync

# Descarga e instala el paquete
```

Uso básico

```
const prompt = require('prompt-sync')();

# Importa el paquete y lo inicializa

let nombre = prompt("¿Cuál es tu nombre? ");

console.log("Hola, " + nombre + "!");

# Al correr: node archivo.js

# ¿Cuál es tu nombre? Ana

# Hola, Ana!
```

Punto crítico — prompt siempre devuelve string

```
let entrada = prompt("Ingresa tu edad: ");

console.log(typeof entrada);
```

```
# string ← aunque el usuario escriba 25

# Si necesitas operar con números, conviérte:

let edad = Number(entrada);

console.log(typeof edad);

# number

# Si el usuario escribe algo no numérico:

let invalido = Number("hola");

console.log(invalido);

# NaN (Not a Number)
```

- **NaN (Not a Number)** es un valor especial de tipo **number** que representa el resultado de una operación aritmética inválida (por ejemplo convertir 'hola' a número). Paradójicamente, **typeof NaN === 'number'**. Para verificar si algo es NaN usa **isNaN(valor)**.

Ejemplo completo — registro de temperatura

```
const prompt = require('prompt-sync')();

let ciudad = prompt("Nombre de la ciudad: ");

let temp_str = prompt("Temperatura en °C: ");

let temp = Number(temp_str);

let estado;

if (temp < 0) {

    estado = 'Bajo cero';

} else if (temp <= 15) {

    estado = 'Frío';

} else if (temp <= 28) {

    estado = 'Agradable';

} else {
```



```
    estado = 'Caluroso';  
  
  }  
  
  console.log(`${ciudad}: ${temp}°C → ${estado}`);  
  
# Bogotá: 14°C → Frío
```

— Ejercicio rápido 8

Usa `prompt-sync` para pedir el nombre de un producto y su precio sin IVA. Calcula e imprime el precio con IVA (19%) y el valor del IVA por separado. Si el precio ingresado es 0 o negativo, imprime 'Precio inválido' en lugar del cálculo.

Pista: Convierte el precio a `Number()` antes de operar y verifica con un `if` si es `<= 0`

SECCIÓN 7 · EJERCICIOS APLICADOS

Ejercicios aplicados — sistemas reales

Estos ejercicios integran todo lo visto. No tienen solución publicada — el criterio de éxito está en cada enunciado. Usa **prompt-sync** para la entrada y **node archivo.js** para ejecutar.

Ejercicio 1 — Sistema de registro de notas

Construye un programa que pida al estudiante su nombre y luego 5 notas numéricas una por una. Al terminar de ingresar las notas, el sistema debe calcular y mostrar el promedio, la nota más alta, la nota más baja y un mensaje de estado: 'Aprobó el módulo' (promedio ≥ 60) o 'Debe recuperar' (promedio < 60).

- Usar prompt-sync para pedir nombre y cada nota
- Almacenar las notas en variables separadas (sin arrays aún)
- Calcular el promedio sumando las 5 notas y dividiendo entre 5
- Encontrar la nota más alta y más baja usando if
- Mostrar el resumen con template literals en formato legible

Ejercicio 2 — Simulador de caja registradora

El programa pide el precio de 4 productos uno a uno. Al final calcula el subtotal, aplica IVA del 19%, pregunta si el cliente tiene tarjeta de descuento (respuesta: s/n). Si la tiene, aplica un 8% adicional sobre el subtotal antes del IVA. Muestra el ticket final con subtotal, descuento (si aplica), IVA y total a pagar.

- Usar prompt-sync para los 4 precios y la pregunta de descuento
- Convertir cada precio a Number() antes de sumar
- Usar if para verificar si respondió 's' y aplicar el descuento
- Calcular en este orden: subtotal $\rightarrow$ descuento $\rightarrow$ IVA $\rightarrow$ total
- Mostrar todos los valores con \$ y formato claro

Ejercicio 3 — Control de acceso por contraseña

El sistema tiene una contraseña correcta definida como constante (la decides tú). Pide al usuario que ingrese la contraseña. Si es correcta, imprime 'Acceso concedido. Bienvenido.' Si no, permite hasta 3 intentos usando un while. Si agota los intentos, imprime 'Acceso bloqueado. Contacta al administrador.'

- Declarar la contraseña correcta con const
- Usar let para el contador de intentos
- Usar while que corra mientras los intentos sean menores a 3 y no haya acertado
- Comparar con === (no con ==)
- Imprimir después de cada intento fallido cuántos intentos quedan

Ejercicio 4 — Reporte mensual de ventas

El programa pide el nombre del vendedor y las ventas de 6 meses (un número por mes, ingresados uno a uno). Usando un for clásico acumula el total de ventas, identifica el mes de mayor venta y el de menor venta. Al final imprime un reporte con: nombre del vendedor, total acumulado, promedio mensual, mes con mayor venta y mes con menor venta.

- Usar `prompt-sync` dentro de un `for` para pedir las 6 ventas
- Convertir cada venta a `Number()` antes de acumular
- Usar variables separadas para guardar el mayor, el menor y sus posiciones
- Calcular el promedio dividiendo el total entre 6
- Mostrar el reporte final con `template literals`, claro y sin abreviaciones